Getting Started

CodeUI provides a fluent C# interface for constructing Unity UI Toolkit components.

---

1. Creating Components with BaseUI

Attach a `UIDocument` component to a GameObject and inherit from `BaseUI` to construct your interface inside `Generate`:

```
using CodeUI;
using CodeUI.Extensions;
using CodeUI.Tokens;
using UnityEngine;
using UnityEngine.UIElements;

public class SettingsScreen : BaseUI
{
    protected override void Generate(VisualElement root)
    {
        root.Column()
            .Padding(Spacing.MD)
            .WithChildren(
                new Label("Settings").FontSize(FontSize.LG),

                new Toggle("Mute Audio")
                    .OnValueChanged<Toggle, bool>(evt => AudioListener.pause
= evt.newValue),

                Spacer.Vertical(Spacing.MD),

                new Button(() => RebuildUI())
                    .SetText("Refresh")
            );
    }
}
```

---

2. Layout & Positioning

CodeUI provides shorthand extension methods for standard Flexbox properties:

Method	Description
`Row()`	Sets flex direction to horizontal (`FlexDirection.Row`)
`Column()`	Sets flex direction to vertical (`FlexDirection.Column`)
`Center()`	Centers elements horizontally and vertically (`Align.Center` , `Justify.Center`)
`Absolute()`	Sets position mode to absolute (`Position.Absolute`)
`Relative()`	Sets position mode to relative (`Position.Relative`)

---

3. Styling & Tokens

Standard design tokens ensure consistent spacing, typography, and layout boundaries across components:

- **Spacing**: `Spacing.XS` (4px), `Spacing.SM` (8px), `Spacing.MD` (16px), `Spacing.LG` (24px), `Spacing.XL` (32px), `Spacing.XXL` (48px)
- **FontSize**: `FontSize.XS` (12px), `FontSize.SM` (14px), `FontSize.MD` (16px), `FontSize.LG` (20px), `FontSize.XL` (28px), `FontSize.XXL` (40px)
- **Radius**: `Radius.None` (0px), `Radius.SM` (4px), `Radius.MD` (8px), `Radius.LG` (16px), `Radius.Full` (999px)

---

4. Event Handling

Events can be attached directly to visual elements during instantiation:

```
button.OnClick(() => Debug.Log("Button clicked"))
      .OnHover(isHovering => button.ToggleClasses(isHovering, "hovered"));
```

---

5. Animations & Transitions

Smooth CSS transitions can be defined using the type-safe `TransitionProperty` enum:

```
card.Transition(TransitionProperty.scale, durationSeconds: 0.2f, EasingMode.EaseOutCubic)
    .OnHover(hovered => card.Scale(hovered ? 1.05f : 1.0f));
```

Data Bindings

Unity UI Toolkit supports Data Binding to automatically synchronize model properties with UI controls. CodeUI provides type-safe lambda expression extensions that eliminate string-based property path errors.

---

1. The Problem with String-Based Bindings

Traditional UI Toolkit bindings rely on magic strings to reference property names on data sources:

```
// Error-prone: Hardcoded string property paths
label.SetBinding("text", new DataBinding
{
    dataSourcePath = new PropertyPath("PlayerName"),
    bindingMode = BindingMode.ToTarget
});
```

Why String-Based Bindings Are Problematic

- **No Compile-Time Verification**: A typo in `"PlayerName"` (such as `"Playername"` or `"Player_Name"`) compiles without warnings and fails silently at runtime.
- **Refactoring Vulnerability**: Renaming properties in your ViewModel or ScriptableObject breaks string paths across your project without triggering IDE refactoring tools.
- **No Type Safety**: The compiler cannot verify whether the source property type matches the target UI element property type.

---

2. Strongly-Typed Lambda Bindings in CodeUI

CodeUI replaces string paths with C# Expression Lambdas (`Expression<Func<TData, TValue>>`). The compiler extracts property names directly from code expressions, providing full compile-time safety and automatic refactoring support.

Text Binding (`BindText`)

Use `BindText` to bind the text of any `TextElement` (e.g., `Label`, `Button`) to a string property:

```
// Strongly-typed string binding
label.BindText(playerData, data => data.PlayerName);
```

Value Binding (`BindValue`)

Use `BindValue` for interactive controls implementing `INotifyValueChanged<TValue>` (e.g., `Toggle`, `Slider`, `TextField`, `IntegerField`):

```
// Two-way binding for interactive controls
toggle.BindValue(settingsData, s => s.MuteAudio);

slider.BindValue(playerData, p => p.VolumeLevel);
```

By default, `BindValue` uses `BindingMode.TwoWay`, meaning user edits in the UI update the data source immediately, and model changes update the UI.

---

3. General Property Binding (`SetUnityBinding`)

For custom properties or non-standard bindings, use `SetUnityBinding` with target and source lambda expressions:

```
// Bind custom property (e.g., progress bar value)
progressBar.SetUnityBinding(
    targetProperty: bar => bar.value,
    dataSource: playerData,
    applyDataSource: true,
    sourceProperty: data => data.HealthRatio,
    mode: BindingMode.ToTarget
);
```

---

4. Assigning Data Sources (`SetUnityDataSource`)

Data sources can be assigned directly to individual elements or to a root container:

```
// Set data source on root container
root.SetUnityDataSource(viewModel);
```